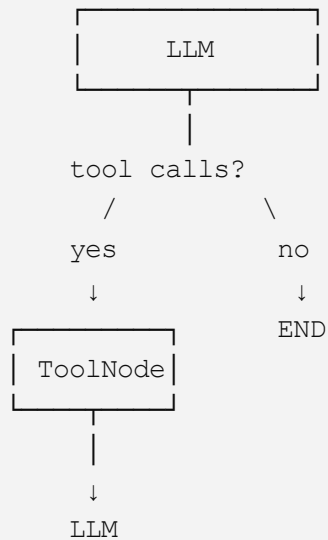


Yep. Now we're moving to the next important piece: **multiple tools + tool calling behavior**.

You already understand the basic loop:

```
LLM → ToolNode → LLM → END
```

Now we want:



But the LLM might request **multiple tools at once**.

1. Create multiple tools

Let's create three:

Python

Run

```
from langchain_core.tools import tool

@tool
def get_weather(city: str) -> str:
    """Get the current weather for a city."""

    weather = {
        "lagos": "Sunny, 30°C",
        "london": "Cloudy, 15°C",
        "new york": "Rainy, 12°C",
    }

    return weather.get(
        city.lower(),
```

```

        f"No weather data available for {city}"
    )

@tool
def calculator(a: float, b: float, operation: str) -> float:
    """Perform a basic mathematical calculation."""

    if operation == "add":
        return a + b

    if operation == "subtract":
        return a - b

    if operation == "multiply":
        return a * b

    if operation == "divide":
        if b == 0:
            return "Cannot divide by zero"

        return a / b

    return "Unknown operation"

@tool
def get_population(city: str) -> str:
    """Get the approximate population of a city."""

    populations = {
        "lagos": "Approximately 15 million",
        "london": "Approximately 9 million",
        "new york": "Approximately 8 million",
    }

    return populations.get(
        city.lower(),
        "Population data unavailable"
    )

```

Now:

Python

Run

```

tools = [
    get_weather,
    calculator,
    get_population,
]

```

2. Bind all of them

Python

Run

```
llm_with_tools = llm.bind_tools(tools)
```

The LLM now has access to:

```
get_weather  
calculator  
get_population
```

It can choose which one to use.

3. What happens with a complex request?

Suppose the user asks:

```
What is the weather in Lagos and what is 25 * 4?
```

The LLM can produce something conceptually like:

Python

Run

```
AIMessage(  
    tool_calls=[  
        {  
            "name": "get_weather",  
            "args": {  
                "city": "Lagos"  
            }  
        },  
        {  
            "name": "calculator",  
            "args": {  
                "a": 25,  
                "b": 4,  
                "operation": "multiply"  
            }  
        }  
    ]  
)
```

Notice:

There are two tool calls inside one AI message.

That's important.

4. ToolNode handles this

You don't need:

Python

Run

```
get_weather.invoke(...)
calculator.invoke(...)
```

manually.

You have:

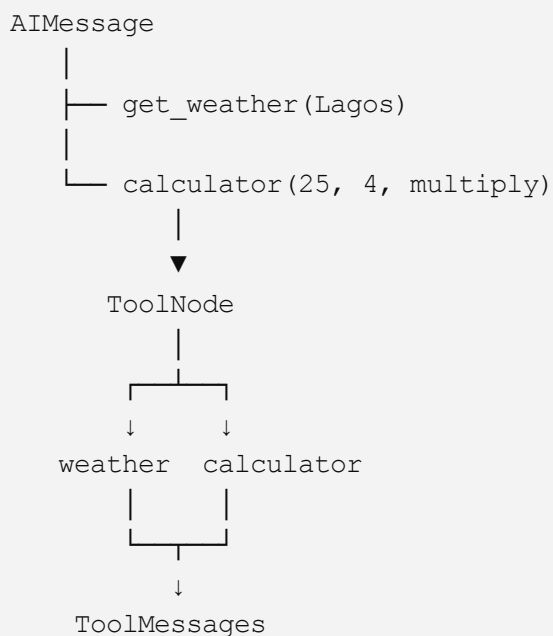
Python

Run

```
tool_node = ToolNode(tools)
```

`ToolNode` looks at the tool calls and executes the appropriate tools.

Conceptually:



Then both results are returned to the graph state.

5. The state after ToolNode

You might have:

messages:

HumanMessage

"What is the weather in Lagos and what is 25×4 ?"

AIMessage

```
[
    get_weather("Lagos"),
    calculator(25, 4, "multiply")
]
```

ToolMessage

"Sunny, 30°C"

ToolMessage

"100"

Then:

Python

Run

```
builder.add_edge("tools", "agent")
```

sends everything back to the LLM.

6. The LLM produces the final answer

Now it has:

Weather:

Sunny, 30°C

Calculation:

100

So it can respond:

Lagos is sunny at 30°C, and $25 \times 4 = 100$.

No more tool calls.

Therefore:

Python

Run

```
should_continue(...)
```

returns:

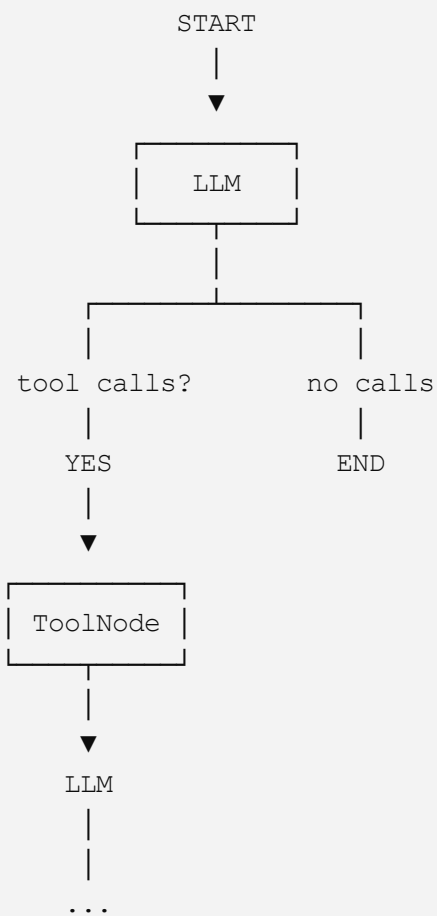
Python

Run

END

7. Your graph is becoming a real agent

The architecture is now:



And this is where the word **agent** starts making sense.

The LLM is not following one fixed path.

It is dynamically deciding:

Should I call a tool?
Which tool?
What arguments?
Do I need another tool?
Am I finished?

8. But there's an important problem

Imagine the user says:

"Calculate 100 / 0."

Your tool currently returns:

Cannot divide by zero

That's fine.

But imagine a tool actually throws:

Python

Run

```
raise ValueError("Something went wrong")
```

Then what happens to your graph?

Potentially:

```
LLM
↓
ToolNode
↓
🔥 ERROR
```

The agent stops.

That's why the next major concept after agent loops is:

Human-in-the-loop + error handling

But before that, there's one foundational concept we need to properly understand:

Tool calling itself

You should be comfortable with:

Python

Run

```
llm.bind_tools(tools)
```

and:

Python

Run

```
response.tool_calls
```

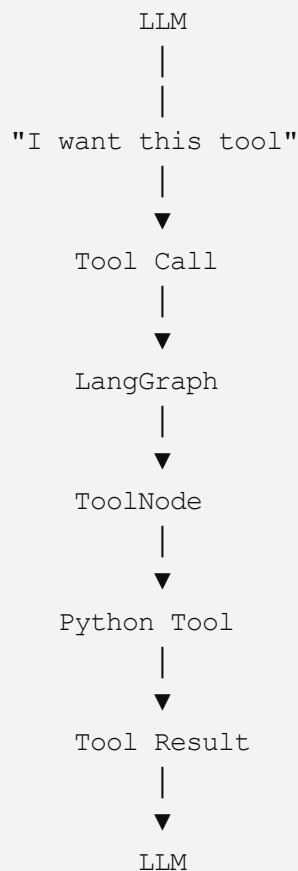
because these are the bridge between the **LLM** and **LangGraph**.

9. The mental model I want you to keep

Don't think:

"The LLM calls my Python function."

Think:



That's the architecture.

Where we go next

You've now covered:

- Tools
- `@tool`
- `bind_tools()`
- Tool calls
- `ToolNode`

- Multiple tools
- Agent loops

So the next topic in your roadmap is:

Human-in-the-loop

This is where agents get much more interesting.

We'll build:

```
User
  ↓
Agent
  ↓
"Should I perform this action?"
  ↓
|| INTERRUPT
  ↓
Human approves/rejects
  ↓
Agent continues
```

And this will naturally introduce **interrupts, persistence, and checkpointing**, which are the next big LangGraph concepts.